

Testril demo plan

Review summary · 3 October 2026

What we should build

Start with **one treasury web application and two small command-line examples**, released as three independent source downloads. The website and recordings show them together; developers can choose the example they need. The demonstrations below describe capabilities, not separate applications for every feature.

Testril charges for preparing and storing computed data (**materialization**), then reading it. The materialization payer can earn from later reads. **Provenance** identifies the sources; independent recalculation checks the result.

What the main application looks like

The treasury application answers “Where did this treasury’s USDC go?” The input form asks for a treasury address, token, block range, optional team addresses, and spending limit. The resulting page contains a balance summary, transfer diagram, ranked counterparties, costs, and expandable source details.

Illustrative result using synthetic data: opening balance **1,000 USDC**, incoming **50**, outgoing **100**, closing **950**, change **–50**. The counterparty table shows outgoing **100 to B**, incoming **30 from B**, and incoming **20 from C**. A self-transfer is excluded from those totals. Reconciliation shows that the balance change matches incoming minus outgoing.

Clicking a number opens its contributing rows and source blocks. **Verify** independently fetches the reference data and recalculates the totals; changing an amount causes a visible mismatch. The page displays actual preparation and read charges separately. An earnings panel shows what the materialization holder earned when the separate reader purchases the same covered data.

See the [proposed screen](#) and [download specifications](#). The illustration is a design proposal, not a live Testril result.

The three first-release downloads

1. Token activity script (P0). A terminal table shows each checked block range, transfer count, read charge, and remaining allowance. The final output identifies activity above a chosen threshold. Developers get a small payment client and a clear rule for choosing the next range. First modification: change the token or threshold. Uses existing transfer volume; no new function is needed.

2. Treasury application (P1 + P3, with the P2 supplier view). Download the web application, its command-line client, calculation code, and verifier. First modification: analyze another treasury or change the supplied team addresses. Requires **F01 transfer edges**, which returns amounts between pairs of addresses, and exact balance snapshots. Verification and earnings are parts of this application.

3. Paid reader script (P2). A second account purchases data already prepared by the supplier. The terminal shows its result, charge, and receipt; the supplier’s earnings panel shows the corresponding increase. The script accepts a bound-function identifier and range and runs without the treasury application. First modification: read another covered interval. Label operator-funded traffic and verify any payout shown.

What developers download

Download source provides a versioned archive with source, a terse README, pinned dependencies, placeholder configuration, sample data, and all required assets. Each archive must run after extraction into an empty directory, without sibling projects or repository-level tooling.

Export this run provides the inputs, returned data, calculated report, receipts, function identity, source references, and verification instructions for that particular result. It contains no credentials. Data exports and source downloads have different purposes and separate controls.

Demos after function creation

Build a separate **vault analysis application** when creation on request works. Its screen contains a definition editor, generated code and tests, quotes, and a result table. G1 classifies deposits by the owner's token balance at the previous block. G2 defines repeat deposits on a later UTC day within seven days and compares revised definitions; incomplete observation periods remain explicit.

Developers download the application and exported generated definitions, with their versions and required runtime. Include a small consumer that reuses a generated function from another client. G3 combines that reuse with materialization earnings. These require vault functions, generation support, and, for G2, stored state and timestamps. Selecting a prewritten function is not creation on request.

Pool analysis (P4) and a REST/card gateway (P5) are optional separate projects. Add them when user trials show a need; keep their setup out of the treasury download.

Small payments and wallet setup

Make the charge for each operation visible. At observed rates, fifty 100-block cached reads would total **\$0.006 (0.6¢)**. Preparing 5,000 distinct blocks would add **\$0.60**. These are illustrative data charges, not a measured competitor benchmark. Use only useful checks and prefer a bulk read when the whole required range is known.

Testril starts with tiny purchases; familiar APIs and billing can sit above them. Competitors have added machine payments to existing APIs, with published cent-scale prices. [Pricing sources](#).

Provide local sample mode, recordings, sponsor-funded live trials, and user-funded access. Sample mode lets developers see the expected output before obtaining a wallet. Live spending limits are enforced outside the LLM. Show deposits separately from charges; use a wallet or signer, never a private-key web form. Sponsored materialization rights remain with the sponsor unless transferred explicitly.

Card billing and REST

A gateway can give customers API keys and card billing while purchasing through Testril's paid MCP interface. Preserve sub-cent usage records and combine charges for billing. Start with prepaid credit and spending caps; price subscriptions after measuring costs. The first gateway holds materialization rights; any customer rebate is explicit service credit. Gateway cache hits do not create Testril read rewards.

Work and release criteria

Build the three first-release downloads before the vault application. Preliminary estimate: **12–23 engineer-days**, including F01 if the engine supports its output and time for standalone archives and documentation. Backend engine changes are additional. Verify paid execution, exact fields, payment network, latency, and payouts on the demo deployment.

Ask five external developers to extract an archive, run its sample, configure live access, and make the suggested modification. Record failures and repeat use. Before public distribution, choose an explicit license and review the release contents and history for secrets.

Decisions needed: choose the real treasury example, assign F01, select the payment network and budget, agree gateway earnings ownership, and define function-creation acceptance tests.

[Detailed plan](#) · [Download specifications](#) · [Function requirements](#)